

## Practical NO. 02

Aim: study of DDA & Bresenham algorithm.

Theory:

1) DDA (Digital differential Analyzer)

- it is a simple and efficient method of drawing lines on a digital screen.
- it is based on the concept of calculating intermediate points b/w the two end points of a line segment and drawing those points.

Algorithm:

1) Read starting and ending points

$(x_1, y_1)$  &  $(x_2, y_2)$

2) calculate

$$dx = x_2 - x_1$$

$$dy = y_2 - y_1$$

3) if ( $dx > dy$ )

length = absolute ( $dx$ )

else

length = abs( $dy$ )

4) Increment factor.

$$\Delta x = dx / \text{length}$$

$$\Delta y = dy / \text{length}$$

5) Plot ( $x_1, y_1$ )

6) while ( $i \leq \text{length}$ )

{

$$x = x + \Delta x$$

$$y = y + \Delta y$$

Plot ( $\text{int}(x), \text{int}(y)$ )

}.  
1  
2  
3  
4  
5



## Bresenham's Line Generation:-

- 1) Plot initial point
- 2) find  $dx, dy, 2dy, 2dy - 2dx$ .

where

$$dx = x_2 - x_1$$

$$dy = y_2 - y_1$$

- 3) initial error function

$$P_0 = 2dy - dx$$

- 4) start iteration with  $k=0$

- a) if  $P_k < 0$

$$\text{Plot}(x_{k+1}, y_k)$$

$$P_{k+1} = P_k + 2dy$$

- b) if  $P_k \geq 0$

$$\text{Plot}(x_{k+1}, y_{k+1})$$

$$P_{k+1} = P_k + 2dy - 2dx$$

- 5) Repeat step 4)  $dx$  times.

Aim: - implement Bresenham circle drawing algorithm to draw any object.

• what is Bresenham circle drawing algorithm.

- The Bresenham circle drawing algorithm is an efficient method. to draw a circle using integer calc. avoiding floating point operations.

# steps.

① Read radius of circle  $r$ .

②  $d = 3 - 2r$

③  $x = 0$ ,  $y = r$

④ do {

    Plot  $(x, y)$

$x = x + 1$

    if  $(d < 0)$  then

$d = d + 4x + 6$

    else

$d = d + 4(x - y) + 10$

$y = y - 1$

    }

    while  $(x < y)$

end.

Q The centre points co-ordinates  $(0, 0)$  & radius as 10 draw circle using Bresenham's algorithm



Aim!: To implement following Polygon Filling method i) Flood Fill ii) Boundary Fill using mouse click keyboard interface & menu driven.

Theory:-

Polyline:- it is a continuous line made up of one or more connected line segments.

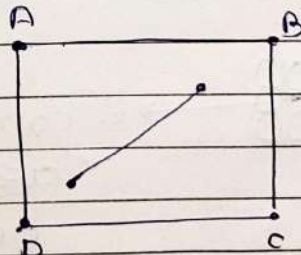
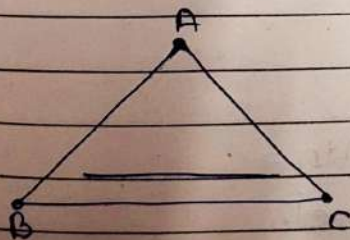
- A Polyline not have to be closed.

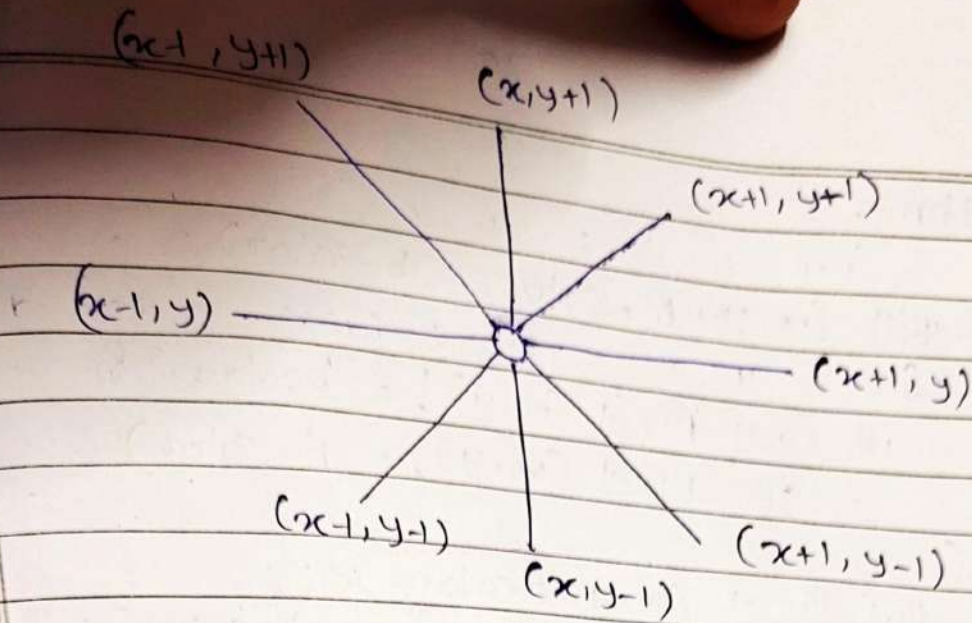
Polygon:- when starting point & terminal point of any Polyline is same i.e. when Polyline is closed then it is called Polygon.

\* Types of Polygon:-

i) convex Polygon:-

- it is A Polygon in which the line segment joining any two points within Polygon lies completely inside the Polygon.





\* Seed Fill Algorithm is used in CG to fill connected region with a specified color. it is commonly used in Paint Programs & image processing.

## # Types of seed fill Algorithm.

### ① Boundary Fill Algorithm:

- fills an area enclosed by a given boundary colour
- stop filling when it reaches the boundary colour.
- works similarly to Flood fill but with a diff stopping condition.



Algorithm:

b-fill ( $x, y, R, colour, b\_color$ )  
{

if ( $get\ Pixel(x, y) \neq b\_colour$  &&

$get\ Pixel(x, y) \neq R\_color$ )

{

Put Pixel ( $x, y, R\_colour$ )

b-fill ( $x+1, y, R\_colour, b\_color$ );

b-fill ( $x, y+1, R\_color - b\_color$ );

b-fill ( $x, y-1, R\_colour, b\_colour$ );

}

}

~~## conclusion:~~

~~we have easy understand.  
the implement polygon filling method~~

~~i) Boundary fill~~

~~ii) Flood fill~~

## ii) Flood Fill Algorithm.

- it is used to fill a connected area of pixels with a certain colour or value, similar to the "bucket fill" tool in image editing software. it is commonly used in ~~6~~ Computer Graphics.

### Algorithm.

- ① Get the original colour of the starting Pixel.
- ② if the original colour is the same as the new colour, return - to avoid infinite recursion)
- ③ use DFS or BFS to visit all connected Pixel of the same colour
- ④ change each visited Pixel's colour to the new colour.
- ⑤ continue until all valid Pixels are killed

Time complexity  $O(N)$

$N \rightarrow$  No. of Pixels to be killed.



Aim: implement the following transformation on object.

- ① Translation.
- ② Scaling.
- ③ Rotation.

Theory:

Transformation: it is the mathematical operation that are used to manipulate the position orientation & size of object in a graphical scene, either in 2D or 3D space. in a graphical scene either in 2D or 3D space often represented using co-ordinates.

Types of Transformation.

① Translation: moving an object to a new location by adding offset to it's co-ordinates transformation

$$\begin{bmatrix} x & y \end{bmatrix} + \begin{bmatrix} tx & ty \end{bmatrix} = \begin{bmatrix} x+tx & y+ty \end{bmatrix}$$

$$\begin{bmatrix} x & y & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ tx & ty & 1 \end{bmatrix}$$

$$= \begin{bmatrix} x+tx & y+ty & 1 \end{bmatrix}$$

② Scaling! changes the size of an object by multiplying this co-ordinates by scaling factors co-ordinated system.

$$\begin{bmatrix} x & y \end{bmatrix} \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} = \begin{bmatrix} xs_x & ys_y \end{bmatrix}$$

- Homogeneous co-ordinates.

$$\begin{bmatrix} x & y & 1 \end{bmatrix} \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} xs_x & ys_y & 1 \end{bmatrix}$$

③ Rotation! rotates an object around a specific Axis.

- Homogeneous co-ordinate system

$$\begin{bmatrix} x & y & 1 \end{bmatrix} \begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} x \cos \theta - y \sin \theta \\ x \sin \theta + y \cos \theta \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} x & y \end{bmatrix} \begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix} = \begin{bmatrix} x \cos \theta - y \sin \theta \\ x \sin \theta + y \cos \theta \end{bmatrix}$$

①  
2nd  
conclusion! Hence we have studied transformation (1) translation 2) scaling 3) rotation) of an object.



## Practical No. 07

Title:- Generate Fractal Patterns using  
i) Bezier ii) Koch curve.

Objective :- To generate Fractal Patterns using  
i) Recursive Subdivision of Bezier curve.  
ii) Koch curve Algorithm.

### ① Bezier curve Based Fractal.

Theory:- it is a curve which is a parametric curve. Frequently used in computer graphics and related field it is defined by four control points  $P_0, P_1, P_2, P_3$  a cubic curve. A Fractal can be generated by recursively subdividing the curve using De Casteljau's Algorithm.

### \* Algorithm.

① Input :- Control Points  $P_0, P_1, P_2, P_3$  and depth of recursion  $n$

② If  $\text{depth} = 0$  draw the Bezier curve from  $P_0$  to  $P_3$

③ calc. intermediate points.

$m_0 = \text{midPoint}(P_0, P_1)$

$m_1 = \text{midPoint}(P_1, P_2)$

$m_2 = \text{mid Point } (P_1, P_3)$

$Q_0 = \text{mid Point } (m_0, m_1)$

$Q_1 = \text{mid Point } (m_1, m_2)$

$R = \text{mid Point } (Q_0, Q_1)$

④ Recursively draw two new curves.

Left:  $P_0, m_0, Q_0, R$ .

Right:  $R, Q_1, m_2, P_3$ .

⑤ Repeat until base case is reached.

⑥ stop.

Ex  $\rightarrow$

Given,

$P_0 = (0, 0), P_1 = (1, 2), P_2 = (3, 2), P_3 = (4, 0)$

Step ①  $\rightarrow$

$$m_0 = \left( \frac{0+1}{2}, \frac{0+2}{2} \right) = (0.5, 1)$$

$$m_1 = (2, 2)$$

$$m_2 = (3.5, 1)$$

Step ②

$$Q_0 = (1.25, 1.5)$$

$$Q_1 = (2.75, 1.5)$$



step ②  $R = (2, 1.5)$

Now we split curve.

Left + bezier :-  $(0, 0), (0.5, 1), (1.5, 1.5), (2, 1.5)$

Right -||- :-  $(2, 1.5), (2.75, 1.5), (3.5, 1), (4, 0)$

These can be further divided recursively to form a fractal.

② Koch Curve.

— it is a classic fractal. constructed by recursively altering line seg. At each iteration every straight line is replaced by four seg. forming a bump shaped like an equilateral triangle.

\* Algorithm

① Input :- start point A, end point B, recursion : depth D

② if  $n = 0$  draw line AB.

③ Else:- divide into three parts C & D

— construct an equilateral triangle with base.

① let the peak be E

Recursively, apply the algorithm to:

A  $\rightarrow$  C

C  $\rightarrow$  E

E  $\rightarrow$  D

D  $\rightarrow$  B.

Example :-

Given A = (0,0) B = (9,0)

Step ① C = (3,0)

D = (6,0)

Step ② CD = 3 units  $\rightarrow$  height.

$$= \frac{\sqrt{3}}{2} \times 3.$$

$$\approx 2.6$$

mid Point of CD = (4.5, 0)

Peak E = (4.5, 2.6)

Final segments:-

A  $\rightarrow$  C = (0,0) to (3,0)

C  $\rightarrow$  E = (3,0) to (4.5, 2.6)

E  $\rightarrow$  D = (4.5, 2.6) to (6,0)

D  $\rightarrow$  B = (6,0) to (9,0)



Practical No. 08

Title :- Implement animation. principles for any objects.

Theory :-

Animation is the process of creating illusion of motion by displaying a series of individual images or frames in rapid succession each frame differs slightly from previous one, making object appear to move.

Types :-

① Traditional Animation (2D) :-

- Hand drawn frame by frame on paper or covey

② 2D Animation (Digital)

- created using computer software like adobe Animate.

③ 3D - Animation

- uses software like Blender, Maya  
- characters & objects move in a 3D environment

ex:- Toy, story.